

Experimental Analysis of Matrix Multiplication Functional Units

Brian Hickmann
Intel Corporation
Hillsboro, OR
brian.j.hickmann@intel.com

Dennis Bradford
Intel Corporation
Portland, OR
dennis.bradford@intel.com

Abstract—The rapid growth of AI has led to the introduction of several new hardware designs to accelerate the matrix multiplication operation at the heart of AI applications. Examples include NVIDIA’s Tensor Core*, Google’s TPU*, and Intel’s Neural Compute Stick*. However, the IEEE 754 standard gives significant implementation-specific flexibility in the definition of the matrix multiplication operation and the precision and compatibility of these new accelerators is not well documented. This paper describes a method exploiting the rounding modes and other features of the IEEE 754 standard in order to gain deeper insight into the design and functionality of matrix multiplication units. We apply this method to the NVIDIA V100 GPU Tensor Core* units and report our findings on the design properties and micro-architecture.

Keywords—Machine learning, deep learning, tensor, half precision, single precision, Volta, matrix multiplication

I. INTRODUCTION

The rapid growth of artificial intelligence and deep learning applications has led to an increased interest in accelerating these workloads. Today’s deep learning workloads are commonly processed using IEEE Standard 754-2008 binary32 (single precision or FP32) or binary16 (half precision or FP16) formats [1], with an increased focus on using the smaller FP16 format or other similar 16-bit floating point formats to reduce memory and computational overheads. The neural networks used by deep learning also rely heavily on matrix operations, especially the matrix multiplication operation, to perform both training and inference of these networks [3].

To accelerate these operations, several companies have produced hardware matrix multiplication designs tailored to deep learning applications. Examples of such accelerators include NVIDIA’s Tensor Core functional units on its most recent GPUs [2], Google’s Tensor Processing Unit (TPU), and Intel’s Neural Compute stick, among others. However, in general these designs have vaguely documented micro-architectures, precision, and order of operations. This is partly due to the implementation-specific flexibility given by the IEEE 754-2008 standard’s definition of the matrix multiplication operation. While the individual multiplication operation is well defined by the standard, the reduction summation of all products is implementation specific. The IEEE 754-2008 Standard specifies that reduction operation may associate in any order or in a wider format, with

differing exceptions allowed based on implementation, as long as the computation avoids underflow and overflow [1].

It is possible, however, to exploit idiosyncrasies in the IEEE 754 rounding definitions and other features of the standard to gain significant insight into the design of a given accelerator. In particular, during the reduction summation of the products, the selection of rounding points and ordering of operations can be exploited to better understand the microarchitecture of the design. By carefully selecting input values and analyzing the rounding of results, we created a testing methodology that can be used to investigate an unknown matrix multiplication design. In this paper, we used this method to investigate the NVIDIA V100 GPU Tensor Core* units. While NVIDIA has published a whitepaper on the architecture of the V100 “Volta” GPU, it contains minimal technical information on the design, simply describing the Tensor Core as computing “64 floating point FMA operations per clock” [2]. With this testing methodology, we were able to explore three aspects of the Tensor Core design, including IEEE standard compliance, hardware micro-architecture, and internal precision.

II. BACKGROUND AND METHODOLOGY

The key insight we used to create our testing methodology is that the order of operations and placement of rounding operations within the hardware design could be exploited to gain significant insight into the design from the user visible results of the operation. This is unlike many existing hardware floating point accelerators, which are more tightly specified by the IEEE 754 standard and typically contain only one rounding step. We created a series of experiments that first confirmed a number of baseline properties of the design, including exception behavior, rounding mode support, and subnormal value support. We then explored the smallest hardware design unit that could be identified, either from documentation or experimentation. Finally, we expanded our experiments to explore the top-level interconnection of these units to create the complete matrix multiplication unit.

We applied this methodology to investigate the Tensor Core microarchitecture. From the whitepaper available describing this design and from the description of the “wmma” instruction used to access the Tensor Core units from NVIDIA’s CUDA toolkit [4], we know that the design’s

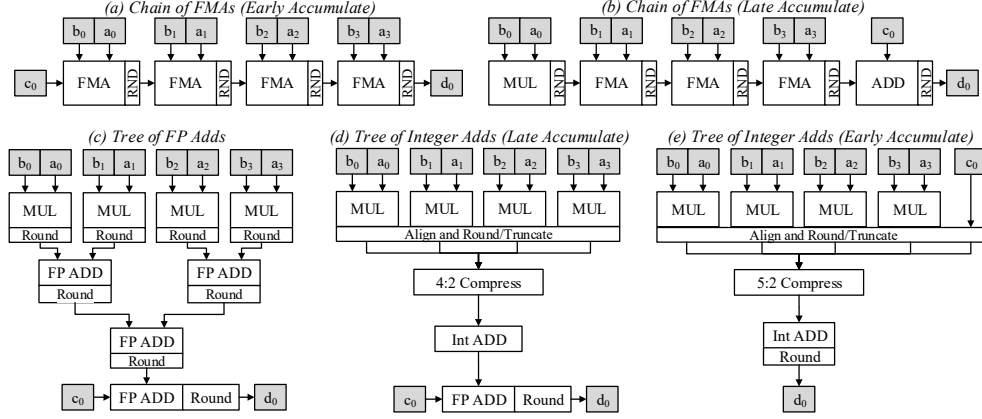


Figure 1. Possible Matrix Multiplication Architectures

base hardware unit performs 4x4 matrix multiplication. We focused one element of the result matrix, which is a 4-input dot product and accumulation in the form:

$$d_0 = (a_0 * b_0) + (a_1 * b_1) + (a_2 * b_2) + (a_3 * b_3) + c_0 \quad (1)$$

Multiplicands a_n and b_n only support FP16 while accumulators d_n and c_n can be stored in either FP16 or FP32 format. In the CUDA ISA spec for the “wmma” instruction, the further description is given that multiplication “is performed with at least single precision” and accumulation is performed “with at least single precision” for FP32 and “with at least half precision” for FP16 [4]. These 4x4 Tensor Cores are combined to perform the matrix operation $D = A * B + C$, with a variety of supported sizes. We focused on the 16x16 matrix sizes exposed by the “wmma” instruction for our testing and used default settings for rounding modes.

While little detail is given in the NVIDIA documentation for the Tensor Core microarchitecture, the fundamental operation can likely be organized in only one of several different ways. We initially drew up these different possible implementations, shown in Figure 1. Figure 1 (a) and (b) show implementations that use FMA units in a chain to perform the dot product, as suggested by NVIDIA’s description of “64 FMAs per clock”. Figure 1 (c) shows an implementation that uses independent Floating Point (FP) multipliers and a tree of FP adders. Finally Figure 1 (d) and (e) show implementations with independent FP multipliers followed by a block alignment and integer reduction tree. The primary differences between these options were the number and placement of the rounding operations, and the early or late addition of the accumulator. Following our testing methodology, we crafted test vectors to highlight these differences and used a small CUDA program to execute our tests on the Tensor Core hardware. The results of this program were then analyzed and multiple testing iterations performed to better understand the design

III. FINDINGS

We have grouped our findings into 3 groups based on our testing methodology: Baseline Design Properties, Internal Dot-Product Datapath, and Top-level Tensor Core Datapath.

A. Baseline Design Properties

We began our testing by confirming the baseline properties of the design, including IEEE 754 standard compliance, rounding mode support, subnormal support, and exception behavior, using straight forward test vectors. Our findings are summarized below.

- 1) *NaN Input Payloads are Not Propagated and Fixed qNaN are Produced:* In all cases of input NaNs or in the exceptional cases of $(0 * \text{Inf})$ and for $(-\text{Inf} + +\text{Inf})$, the Tensor Core design produces the qNaN indefinite with the encoding 0x7fff or 0x7fffffff.
- 2) *Output Infinities are Produced with Correct Sign:*
- 3) *FP16 Input and Output Subnormals are Correctly Computed and not Treated as Zero:*
- 4) *FP32 Input Subnormals are Treated as Zero:*
- 5) *Sign of Zero Output is Always Positive:* In the case that all the products were -0.0, the result was always +0.0.

B. Internal Dot-Product Datapath

1) *Internal Datapath is Not a Chain of FMAs or a Tree of FP Adders:* If the internal datapath was a chain of FMAs or a tree of FP adders, we have internal rounding points that would produce different results depending on the order of the operands. To expose this, we selected multiplier inputs to produce $2^{30} + -2^{30} + 2^{-14}$ and then permuted these inputs across all possible locations in the A and B matrices. This “Big + -Big + Small” test vector was chosen to highlight the differences in rounding between possible implementations. If there were internal rounding points in a tree or chain, we would obtain either 0.0 or 2^{-14} as the result. We would receive an answer of 2^{-14} if the $2^{30} + -2^{30}$ were done first, as the result of this sub-operation would be exactly zero. We

would receive an answer of 0.0 if the $2^{-14} + 2^{30}$ was done first, as difference in exponents here is so large that 2^{-14} would have been rounded away, producing 2^{30} . When added to -2^{30} this would produce 0.0. However in our testing, the result was always 0.0, which indicates there is no internal rounding point that depends on the order of operations.

2) *Internal Datapath has a Fixed Width Truncated to 24 Bits:* Based on the previous experiment, we expect that similar to Figure 1 (d) or (e), all products are first aligned to the maximum exponent and then added using a datapath with a fixed internal width. To determine the width of the datapath, we ran a series of tests performing $2^{30} + -2^{30} + 2^N$, where N varied from -28 to 30. Since we are expecting all values to be aligned initially, this 2^N value would be rounded or truncated off if it is shifted off to the right of the datapath width. As we varied N, we found that the datapath had an internal width of 24 bits, as we saw the result change from 0.0 with N=6 to 2^7 when N=7. This 24-bit datapath width matches the instruction description of “accumulating with at least single precision” from NVIDIA’s documentation [4].

We also tested what happens when we have an integer overflow from the mantissa addition. We performed $1.0 + 1.0$ to produce an integer overflow bit and added $2^{-22} + 2^{-23}$ to the result. In this case, the final 2^{-23} becomes the 25th bit of the infinitely precise result. If this bit was saved or used for rounding, we would expect this result to have rounded up for some rounding modes. However in all cases, this bit was truncated off, producing “01” in the least significant bit positions (LSBs) of the result instead of “10” which we would expect if a round up took place.

3) *FP32 Result from a Single Dot-Product Datapath is Truncated, Not Rounded:* Having determined the width of the datapath, we tested which rounding mode was used to form the final result. We started with FP32 and performed a basic rounding test of $1.0 + 2^{-23} + 2^{-24}$ as well as the negative version of the same test. If an IEEE 754 rounding mode other than truncation were used, one of these two tests would have rounded up to produce “10” in the two LSBs. However in both tests, the result contained “01” in the last two bit positions, indicating the result was truncated.

4) *FP16 Results from a Single Dot-Product Datapath are Rounded Using IEEE 754 Round to Nearest Even Rounding:* To test rounding of the FP16 results, we ran a similar test with $1.0 + 2^{-10} + 2^{-11}$. In this case, for both positive and negative versions of the test, we see a round up of the result to have “10” in the LSBs. This matches the behavior expected for IEEE 754 Round to Nearest Even (RNE) rounding.

5) *FP16 Output Mode Uses the FP32 Datapath:* In order to test what datapath width was used in FP16 mode, we performed a sweep of tests performing $1.0 + 2^{-11} + 2^N$, with N ranging from -30 to -12. In this case, 2^N is acting as the sticky bit in the RNE calculation. We expected to not see a round up if the 2^N bit was outside the datapath width and

we expect to see a round up in the case that the 2^N was within the datapath width. Once again we see this transition happen at the 24-bit barrier, just as with the FP32 tests run earlier. This implies that the FP16 calculation simply uses the same datapath as the FP32 calculation and the result is rounded to FP16 width at the output.

C. Tensor Core Interconnection

To test how these Tensor Core units were connected together to perform larger matrix multiply operations, we expanded our test vectors to encompass all 16 elements of the A and B operands (as a reminder, CUDA software exposes 16x16 matrices to the user). We continued to focus on a single element of the C and D matrices to limit our scope, and created input permutations across the different 4x4 units. We will describe the result of each Tensor Core as T_n , with $n = 0..3$, where:

$$T_n = [a_{4n}, a_{4n+1}, a_{4n+2}, a_{4n+3}] * [b_{4n}, b_{4n+1}, b_{4n+2}, b_{4n+3}] \quad (2)$$

1) *C Matrix is Added to the Dot-Product Result Using IEEE 754 RNE Rounding:* To test how the accumulator was introduced, we performed the operation $1.0 + 2^{-22} + 2^{-23}$ within a single dot-product operation. This complete result is contained within an FP32 mantissa. We then added a C accumulator value of 1.0, which pushes the LSB of the result off the edge of the FP32 mantissa and causes rounding. In both positive and negative versions of this test, we did see a round up consistent with the RNE rounding mode. We also tested these values across all possible permutations of the A and B input matrices and the results were consistent. Similarly, we tested with similarly constructed FP16 values and obtained the same results. Our conclusion is that the C matrix values are always added to the result of the Tensor Core using RNE rounding, which excludes the possibility of the microarchitecture found in Figure 1 (e) being used to add the C value.

2) *Input 16x16 A and B Matrices are Split Into 8 Element Groups:* To investigate how Tensor Cores are connected, we began with a simple test of $2^{30} + -2^{30} + 2^{-14}$ as we did with the internal testing, but we permuted these three inputs across all 16 elements of the A and B matrices. In this testing, C was always set to 0.0. While testing the FP32 mode, we saw a clear delineation of results depending on if all 3 inputs were in the same group of 8 inputs, (T_0, T_1) or (T_2, T_3), or were split across these 8 input groups.

We continued this testing in FP16 mode which gave further insight into this breakdown. With the same input of $2^{30} + 2^{-30} + 2^{-14}$, we saw the outputs set $\{0, 2^{-14}, +\text{Inf}, -\text{Inf}, \text{NaN}\}$. By analyzing these results for each input permutation, given that we know FP16 results are always rounded using RNE, a distinct ordering of operations emerged. For example, the only way to obtain a NaN as a result was if we had performed $+\text{Inf} + -\text{Inf}$. Thus, we were

able to conclude that the Tensor Core results are added in the following order:

$$(T_0 + T_1) + (T_2 + T_3) \quad (3)$$

3) *For FP32, Neighboring Tensor Core Results are Added Using Truncation but the Final Summation is Done Using RNE. For FP16, All Tensor Cores are Added Using RNE:* To investigate how the summation between Tensor Core results were rounded, we again took the test of $1.0 + 2^{-23} + 2^{-24}$ and permuted these inputs across all 16 elements of A and B. The accumulator input C was 0.0. With these inputs it is clear if truncation rounding was used, as the result will have “01” in the LSBs, or if we have RNE rounding, as the result will have “10” in the LSBs. In this testing, if all three inputs were within Tensor Core groups (T_0, T_1) or (T_2, T_3), the result always used truncation. However, if the inputs spanned across the (T_0, T_1) and (T_2, T_3) boundary, the result was obtained using the RNE rounding mode.

Similar to our FP32 testing, we took the FP16 input values of $1.0 + 2^{-9} + 2^{-10}$ and permuted these inputs across all 16 elements of A and B. The accumulator input C was 0.0. In this case, the results of all permutation were always consistent with the use of RNE rounding, indicating all Tensor Cores were added together using this rounding mode.

4) *Accumulator is Added to Result of ($T_0 + T_1$) and Then Added to ($T_2 + T_3$):* To determine how the accumulator addition was ordered in relation to the sum of the Tensor Core units, we took our previous tests and added an accumulator value of 1.0. This gave us input values of $(1.0 + 2^{-23} + 2^{-24}) + 1.0$ for FP32 and $(1.0 + 2^{-9} + 2^{-10}) + 1.0$ for FP16. The addition of another 1.0 to the input values shifts the result mantissa to the right one additional bit. This will cause a possible second rounding of the result value. By analyzing all permutations of the input values across the 16 elements of A and B for this second rounding of the result, we concluded that the ordering must be:

$$D_0 = (C_0 + (T_0 + T_1)) + (T_2 + T_3) \quad (4)$$

IV. TENSOR CORE ARCHITECTURE ESTIMATE

Combining all of our findings for both the internal Tensor Core microarchitecture and external sum of individual Tensor Core results, our best estimate is pictured in Figure 2. Within each Tensor Core dot-product are four multipliers followed by a block alignment step which aligns all 4 products and a 5th addend input to a single exponent. Extra bits are truncated and an integer adder tree performs the summation. The sum is then renormalized, truncated to 24 bits, and either formed into a final FP32 result or rounded to FP16 precision using the RNE rounding mode. The result of each unit is then either recirculated back to the block normalization or passed to a chain of FP Adders which compute the final result, using RNE rounding.

Since T_0 and T_1 are added together first, we concluded that this is likely done by a second pass through the same

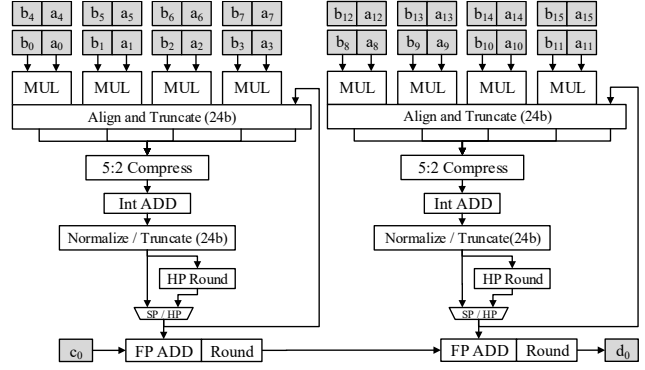


Figure 2. Best Estimate of Tensor Core Dot-Product

hardware as shown. This is further reinforced by the fact that SP results summed within the same group of inputs used truncation and not RNE rounding. A design of this nature would allow easy support for the 8x32, 32x8, and 16x16 matrix sizes exposed by the CUDA “wmma” instruction by varying the number of passes performed. However, we did not test other matrix sizes. Also, while separate FP Adders are shown in the diagram, these could easily be reused from other existing FP adders present on the Volta die.

V. CONCLUSION

We described a testing methodology for matrix multiplication units that uses software visible inputs/output to exploit the order of operations and placement of rounding operations within in the hardware to gain insight into the design. We applied this methodology to the NVIDIA V100 GPU Tensor Core units and were able to explore several different properties of the microarchitecture. From this testing, we synthesized a final estimate of the Tensor Core dot-product microarchitecture. In future work, we would like to expand this testing methodology to include other formats, such as bfloat16, and apply it to other designs, such as Google’s TPU.

ACKNOWLEDGMENT

We would like to acknowledge and thank Michael Ferry, Valentin Andrei, and Marian Alin Petre.

* Other names and brands may be claimed as the property of others.

REFERENCES

- [1] “IEEE standard for floating-point arithmetic,” IEEE Std 754-2008, Aug 2008.
- [2] “NVIDIA Tesla V100 GPU Architecture”, Aug 2017 <http://tinyurl.com/volta-architecture-whitepaper>
- [3] I. Goodfellow et al., Deep Learning. MIT Press, 2016.
- [4] “Cuda Toolkit Documentation” <https://docs.nvidia.com/cuda/>